

ЛАБОРАТОРНАЯ РАБОТА №8
ИСПОЛЬЗОВАНИЕ ЭЛЕМЕНТОВ ДЛЯ ОТОБРАЖЕНИЯ ДАТЫ И ВРЕМЕНИ
Что делаем сегодня?

Любое настоящее приложение работает с **датами**. Например: билеты, доставка, запись к врачу, брони. Сегодня сделаем приложение для бронирования столика, а также познакомимся с одной интересной вещью в **ресурсах**.

Основные правила

1. Лабораторная работа разделена на **ЧАСТИ** – выполняете каждую из частей и **показываете**.
2. Обычно в конце каждой части есть **ЗАДАНИЕ** – его надо сделать
3. Не бойтесь **задавать любые** вопросы
4. Если хотите сделать что-то **свое**, то **предлагайте** – будем думать
5. Советую читать текст, тут **написано** много полезного
- 6.



Кanye Уэст обречен жить в тени ANDROID STUDIO — концерта в «Лужниках...

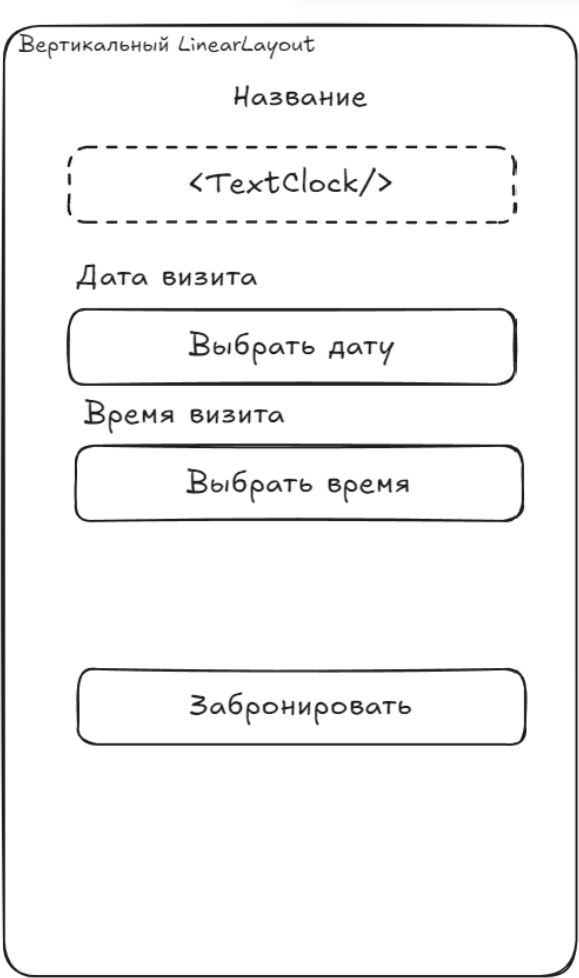
4 часа назад



Neva.Today

Часть 1. Экран и TextClock

Создайте экран по макету:



1. Обратите внимание, там есть новый виджет – **TextClock**:

он показывает текущие дату и время и обновляется сам, ноль строк кода. Формат **задаётся строкой** из букв:

```
<TextClock
  android:id="@+id/textClock"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:layout_marginTop="8dp"
  android:format12Hour="dd.MM.yyyy hh:mm a"
  android:format24Hour="dd.MM.yyyy HH:mm"
  android:textSize="18sp" />
```

Буква	Означает	Пример
dd	день, две цифры	22
MM	месяц, две цифры	09
yyyy	год	2026
HH	час, 0–23	14
hh	час, 1–12	02
mm	минуты	05
a	AM/PM	PM

Два атрибута формата нужны потому, что устройство может работать в 12 или 24 часовом режиме: **format24Hour** для 24-часового, **format12Hour** для 12-часового. Система сама возьмёт нужный.

! ЗАДАНИЕ !

1. вынесите все строки из элементов в ресурсы **strings.xml**.
2. назначьте каждому элементу свой **id**, например:

```
android:id="@+id/btnOrder"
```

3. **Запустите** – дата и время на экране идут.

Часть 2. Дата и DatePickerDialog

Для выбора даты не нужно ничего создавать самому – в Android есть готовый системный **DatePickerDialog**: диалог с календарём на весь экран.

Результат придет в слушателя – прям как у кнопок в: подписываемся, система вызывает наш метод, когда пользователь выбрал дату.

Добавьте в **MainActivity** поля чтоб запомнить дату и время и заполните их:

```
private Calendar selectedDate; // выбранные год, месяц и день 9 usages
private Calendar selectedTime; // выбранные часы и минуты 8 usages

// элементы интерфейса
private TextView tvDate, tvTime, tvResult; 2 usages

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable( $this$enableEdgeToEdge: this);
    setContentView(R.layout.activity_main);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v, WindowInsetsCompat insets) -> {
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
        return insets;
    });

    tvDate = findViewById(R.id.tvDate);
    tvTime = findViewById(R.id.tvTime);
    tvResult = findViewById(R.id.tvResult);
}
```

Теперь добавим в **MainActivity** метод для выбора, сохранения и вывода даты:

```
private void showDatePicker() { no usages
    Calendar today = Calendar.getInstance(); // получаем сегодня

    // создаем слушателя на выбор даты
    DatePickerDialog.OnDateSetListener dateListener =
        ( DatePicker view, int year, int month, int day) -> {
            selectedDate = Calendar.getInstance();
            selectedDate.set(year, month, day); // сохраняем после выбора
            tvDate.setText(day + "/" + month + "/" + year); // выводим
        };

    // создаем диалог
    new DatePickerDialog(context: this, dateListener, // навешиваем слушателя
        today.get(Calendar.YEAR), // ставим по умолчанию текущий год
        today.get(Calendar.MONTH), // текущий месяц
        today.get(Calendar.DAY_OF_MONTH) // текущий день
    ).show(); // показываем диалог
}
```

А после подключим наш метод к кнопке для даты. В конец **onCreate** допишите:

```
findViewById(R.id.btnDate).setOnClickListener( View v -> showDatePicker());  
}
```

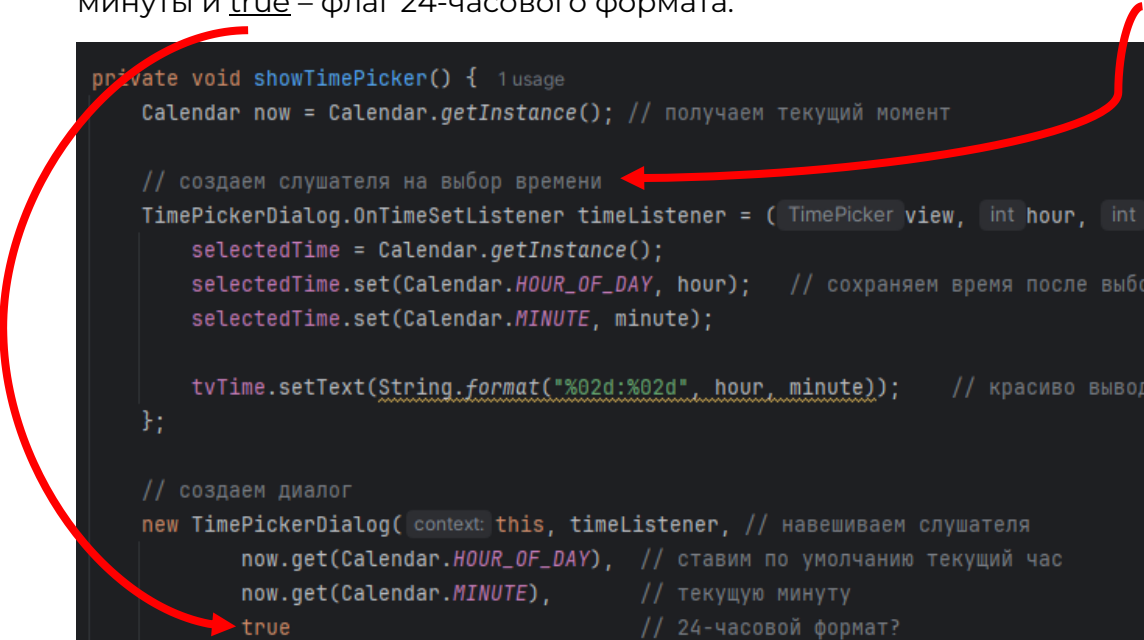
! ЗАДАНИЕ !

Запустите и выберите любую дату. Пусть выберем какое-то число сентября – но на экране выводит, что **сентябрь это 8 месяц**. Исправьте эту ошибку в выводе.

Часть 3. Время и TimePickerDialog

Дата и время по сути устроены одинаково – нам нужен диалог + слушатель + сохранить + показать.

Прежде чем смотреть код ниже, попробуйте написать время сами по образцу из части 2. Конструктору **TimePickerDialog** нужны: контекст, слушатель, часы, минуты и true – флаг 24-часового формата:



```
private void showTimePicker() { 1 usage
    Calendar now = Calendar.getInstance(); // получаем текущий момент

    // создаем слушателя на выбор времени
    TimePickerDialog.OnTimeSetListener timeListener = (TimePicker view, int hour, int minute) -> {
        selectedTime = Calendar.getInstance();
        selectedTime.set(Calendar.HOUR_OF_DAY, hour); // сохраняем время после выбора
        selectedTime.set(Calendar.MINUTE, minute);

        tvTime.setText(String.format("%02d:%02d", hour, minute)); // красиво выводим
    };

    // создаем диалог
    new TimePickerDialog(context: this, timeListener, // навешиваем слушателя
        now.get(Calendar.HOUR_OF_DAY), // ставим по умолчанию текущий час
        now.get(Calendar.MINUTE), // текущую минуту
        true // 24-часовой формат?
    ).show(); // показываем диалог
}
```

(Также не забываем прикрепить этот метод к нашей кнопке выбора времени так же, как делали с датой)

! ЗАДАНИЕ !

Запустите и выберите любую дату и любое время. Попробуйте передать в TimePickerDialog вместо **true** – **false**, что изменится?

Часть 4. Кнопка “Забронировать”

Что должно происходить по нажатию на “Забронировать”? Должно произойти три вещи:

1. проверяем, что **всё выбрано** (и дата, и время)
2. проверяем, что бронируем **не на уже прошедшее время**
3. показываем итоговую **сводку** (с датой, временем и количеством дней до брони)

Добавим в наш **MainActivity** методы для подсчета дней между двумя датами:

```
// метод для вычисления количества дней между датами
private long daysBetween(Calendar from, Calendar to) { 1 usage
    Calendar start = (Calendar) from.clone();
    Calendar end = (Calendar) to.clone();

    setMidnight(start);
    setMidnight(end);

    long day = 24 * 60 * 60 * 1000; // сколько миллисекунд в 1 дне

    return (end.getTimeInMillis() - start.getTimeInMillis()) / day;
}

// обнуление времени для верного подсчета дней
private void setMidnight(Calendar calendar) { 2 usages
    calendar.set(Calendar.HOUR_OF_DAY, 0);
    calendar.set(Calendar.MINUTE, 0);
    calendar.set(Calendar.SECOND, 0);
    calendar.set(Calendar.MILLISECOND, 0);
}
```

Почему работаем именно с миллисекундами? Потому что это самый простой способ сравнить две даты.

getTimeInMillis() возвращает количество миллисекунд с 1 января 1970 года. Разница двух точек, делённая на длину суток = количество дней между ними.

Используем тип **long**, потому что счёт идёт аж с 1970 года: числа там миллиарды, а **int** в итоге не хватает.

Теперь же добавим в **MainActivity** метод для вывода красивого окошка сводки:

```
private void showResultDialog(Calendar now, Calendar visit) { 1 usage
    long days = daysBetween(now, visit);
    // собираем строчку дней
    String daysLine;
    if (days == 0) {
        daysLine = "Бронь сегодня";
    } else {
        daysLine = "До визита: " + days + " дн.";
    }

    // собираем строчку даты
    int day = selectedDate.get(Calendar.DAY_OF_MONTH);
    int month = selectedDate.get(Calendar.MONTH);
    int year = selectedDate.get(Calendar.YEAR);
    String dateStr = day + "." + month + "." + year;

    // собираем строчку времени
    int hour = selectedTime.get(Calendar.HOUR_OF_DAY);
    int minute = selectedTime.get(Calendar.MINUTE);
    String timeStr = String.format("%02d:%02d", hour, minute);

    Log.i(tag: "Booking", msg: "Бронь: " + dateStr + " " + timeStr);
    // выводим уведомление
    new AlertDialog.Builder(context: this)
        .setTitle("Готово!")
        .setMessage("Столик на " + dateStr + " в " + timeStr + ". " + daysLine)
        .setPositiveButton(text: "Закрыть", listener: null)
        .show();
}
```

И по итогу остается всё это дело проверить и вызвать, добавляем новый метод в **MainActivity** (да, опять):

```
private void book() { 1 usage
    // проверяем выбраны ли дата, и время
    if (selectedDate == null || selectedTime == null) {
        tvResult.setText("Сначала выберите дату и время");
        return;
    }

    // собираем из сохраненной инфы ДАТУ бронирования
    Calendar visit = Calendar.getInstance();
    visit.set(
        selectedDate.get(Calendar.YEAR),
        selectedDate.get(Calendar.MONTH),
        selectedDate.get(Calendar.DAY_OF_MONTH),
        selectedTime.get(Calendar.HOUR_OF_DAY),
        selectedTime.get(Calendar.MINUTE),
        second: 0
    );

    // получаем текущий момент
    Calendar now = Calendar.getInstance();

    // проверяем в прошлом ли бронь по сравнению нынешним моментам
    if (visit.before(now)) {
        Log.w(tag: "Booking", msg: "Попытка брони в прошлое");
        tvResult.setText("Нельзя бронировать на прошлое");
        return;
    }

    // показываем наш красивый вывод
    showResultDialog(now, visit);
}
```

И так же не забываем дописать наш **onCreate** чтобы наша кнопка "Забронировать" работала:



```
findViewById(R.id.btnDate).setOnClickListener(View v -> showDatePicker());
findViewById(R.id.btnTime).setOnClickListener(View v -> showTimePicker());
findViewById(R.id.btnBook).setOnClickListener(View v -> book());
}
```

Требования к итоговой версии

1. **TextClock** показывает текущую дату и время
2. Дата и время выбираются системными **диалогами**, месяц на экране **правильный**
3. Бронирование до выбора даты или времени блокируется понятным сообщением
4. Прошлые дата/время **отклоняются** (проверка before())
5. Успешная бронь – **AlertDialog** со сводкой
6. Все строки в **ресурсах**

Дополнительно (по желанию)

1. **Chronometer**. Виджет-секундомер: **<Chronometer>** + chronometer.start() в onCreate – “вы в приложении: 00:03:42”
2. **Часы работы**. Пусть брони будут приниматься только с 10:00 до 22:00. Отклоняйте неподходящее время с понятным сообщением